# PROJECT REPORT

## WEEK 5–7 DELIVERABLES

Prototype Implementation — Smart CPU Scheduler

| | | | |
|---|---|---|---|
| **Course** | Operating System / CSC323 | **Instructor** | Ahmed Abd Elmoaty |
| **Department** | Computer Science | **Phase** | Week 5–7 |

**Prepared by:**

| # | Student Name | Student ID |
|---|---|---|
| 1 | Logain Elsayed | 420230233 |
| 2 | Farah Amr Abdelghafour | 420230654 |
| 3 | Yehya Hany | 420230317 |
| 4 | Mostafa Hany | 420230289 |
| 5 | Mennat Allah Ayman | 420230597 |

# 1. Project Overview

This report documents the Week 5–7 prototype deliverable for the Smart CPU Scheduler project (CSC323). The prototype implements a traditional Round Robin (RR) scheduler in Python, establishing the baseline for comparison with the upcoming Smart Scheduler in Weeks 8–9.

> **Project Goal**
> Build a Smart CPU Scheduler that dynamically adjusts the time quantum based on process type (CPU-bound vs. I/O-bound), improving CPU utilization and reducing average waiting time compared to traditional Round Robin.

# 2. Round Robin Scheduler — Code Explanation

The following section explains the Round Robin scheduler implemented in Python as part of the Week 5–7 prototype. Each part of the code is described in detail.

## 2.1 What is Round Robin Scheduling?

Round Robin (RR) is a CPU scheduling algorithm where each process is assigned a fixed time slot called a time quantum. If a process does not finish within its quantum, it is moved to the back of the ready queue and waits for its next turn. This ensures fairness by treating all processes equally.

| Property | Value / Description |
|---|---|
| Algorithm Type | Preemptive |
| Time Quantum | Fixed (4 units in this prototype) |
| Queue Type | FIFO — First In, First Out |
| Fairness | Equal treatment for all processes |
| Use Case | Time-sharing systems |

## 2.2 Code Walkthrough — Line by Line

**Step 1: Define Processes and Time Needed**

```
# List of process names
processes = ["P1", "P2", "P3"]
# Burst time (CPU time required) for each process
time_needed = [10, 5, 8]
# Fixed time quantum — max time each process gets per round
time_limit = 4
```

We define three processes: P1, P2, and P3. Each has a burst time representing how much CPU time it needs. The time quantum is set to 4 units, meaning no process can run for more than 4 units before being preempted.

**Step 2: The Scheduling Loop**

```
# Outer loop — runs 4 rounds
for round_number in range(4):
print("\n--- ROUND",
round_number + 1, "---")      #
Inner loop — goes through each
process      for i in range(3):
current_process = processes[i]
time_left = time_needed[i]
```

The outer loop simulates rounds of scheduling. In each round, every process gets a chance to run. The inner loop iterates through all processes and checks how much time each one still needs. **Step 3: Preemption Logic**

```
        # If process still has work to do          if time_left > 0:
# If it needs more time than the quantum → preempt it              if
time_left > time_limit:                  print(current_process, "runs
for", time_limit, "minutes.")
                time_needed[i] = time_left - time_limit              # If it
can finish within the quantum → let it finish              else:
print(current_process, "runs for", time_left, "minutes and is
DONE!")
                time_needed[i] = 0
```

This is the core of Round Robin. If a process still needs more time than the quantum allows, it runs for exactly 4 units and gets preempted. Its remaining time is updated. If it can finish, it runs for its remaining time and is marked complete.

> **Key Concept — Preemption**
> Preemption means stopping a running process before it finishes, to let another process run. This is what makes Round Robin fair — no single process can monopolize the CPU.

## 2.3 Sample Output & Trace

Running the code with processes P1=10, P2=5, P3=8 and quantum=4 produces the following execution trace:

| Round | Process | Action | Time Used | Remaining |
|---|---|---|---|---|
| 1 | P1 | Preempted | 4 | 6 |
| 1 | P2 | Preempted | 4 | 1 |
| 1 | P3 | Preempted | 4 | 4 |
| 2 | P1 | Preempted | 4 | 2 |
| 2 | P2 | **DONE ✓** | 1 | 0 |
| 2 | P3 | **DONE ✓** | 4 | 0 |
| 3 | P1 | **DONE ✓** | 2 | 0 |

# 3. Complete Project Code — Smart CPU Scheduler

The following is the complete Python implementation developed in Google Colab. It covers four parts: AI model training, Standard Round Robin, AI Smart Scheduler, and visual performance comparison.

## 3.1 Part 1 — Import Libraries & Train AI Model

We import matplotlib for charts and sklearn for the AI classifier. The DecisionTreeClassifier is trained using process features: [time_needed, io_requests]. Label 0 = I/O-bound, Label 1 = CPUbound.

```
# ==========================================
# 1. TRAIN AI MODEL
# ==========================================
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
```

```
ai_model = DecisionTreeClassifier()
# features: [time needed, I/O requests]  |  0=I/O-bound, 1=CPU-bound
ai_model.fit([[2,10],[3,8],[15,1],[20,0],[5,6],[12,2]], [0,0,1,1,0,1])
```

```
processes = ["P1 (CPU)", "P2 (I/O)", "P3 (CPU)"]
process_features = [[10, 1], [5, 10], [8, 2]]
```

| Sample | Time Needed | I/O Requests | Label |
|---|---|---|---|
| 1 | 2 | 10 | I/O-bound (0) |
| 2 | 3 | 8 | I/O-bound (0) |
| 3 | 15 | 1 | CPU-bound (1) |
| 4 | 20 | 0 | CPU-bound (1) |
| 5 | 5 | 6 | I/O-bound (0) |
| 6 | 12 | 2 | CPU-bound (1) |

## 3.2 Part 2 — Standard Round Robin

The run_standard_rr() function runs classic Round Robin with a fixed quantum of 4. It logs when each process finishes and returns the average turnaround time.

```
# ==========================================
# 2. STANDARD ROUND ROBIN
# ==========================================
def run_standard_rr():     print("---
STANDARD ROUND ROBIN ---")
    time_needed = [10, 5, 8]
    time_limit = 4
current_time = 0
    turnaround_times = [0, 0, 0]
for round_number in range(4):
for i in range(3):
time_left = time_needed[i]
```

```
            if time_left > 0:
if time_left > time_limit:
                    current_time += time_limit
time_needed[i] -= time_limit
else:                     current_time +=
time_left                     time_needed[i] =
0
                    turnaround_times[i] = current_time
                    print(f"{processes[i]} finished at {current_time} mins.")
    return sum(turnaround_times) / 3
```

## 3.3 Part 3 — AI Smart Scheduler

The run_ai_smart_rr() function uses the trained AI model to reorder the queue (I/O-bound to front) and assign dynamic quanta: CPU-bound gets 8 units, I/O-bound gets 5 units.

```
# ==========================================
# 3. AI-BASED SMART SCHEDULER
# ========================================== def
run_ai_smart_rr():     print(" --- AI SMART SCHEDULER
(With I/O Priority) ---")
    time_needed = [10, 5, 8]
    current_time = 0
    turnaround_times = [0, 0, 0]
    # Reorder queue: I/O-bound → front, CPU-bound → back
    smart_order = []     for i in range(3):         if
ai_model.predict([process_features[i]])[0] == 0:
smart_order.insert(0, i)         else:             smart_order.append(i)
# Dynamic quantum: CPU=8, I/O=5     while sum(time_needed) > 0:
for i in smart_order:             if time_needed[i] > 0:
limit = 8 if ai_model.predict([process_features[i]])[0] == 1
else 5
                spent_time = min(time_needed[i], limit)
                current_time += spent_time
time_needed[i] -= spent_time                 if
time_needed[i] == 0:
turnaround_times[i] = current_time
                    print(f"[AI] {processes[i]} finished at {current_time}
mins.")
    return sum(turnaround_times) / 3
```

| Process | AI Prediction | Queue Position | Quantum |
|---|---|---|---|
| P1 (CPU) | CPU-bound (1) | Back of queue | 8 units |
| P2 (I/O) | I/O-bound (0) | Front of queue | 5 units |
| P3 (CPU) | CPU-bound (1) | Back of queue | 8 units |

## 3.4 Part 4 — Run & Visualize Results

Both schedulers are called and their average turnaround times are compared. A bar chart is generated with matplotlib showing the performance gain of the AI scheduler over standard Round Robin.

```
# ==========================================
# 4. SIMULATIONS & VISUALS
# ==========================================
standard_tat = run_standard_rr()
ai_tat       = run_ai_smart_rr()
```

```
print(f"Standard Avg Turnaround Time: {standard_tat:.2f} mins")
print(f"AI Smart Avg Turnaround Time: {ai_tat:.2f} mins")
```

```
labels = ['Standard Round Robin', 'AI Smart Scheduler']
values = [standard_tat, ai_tat]
plt.figure(figsize=(7, 5))
plt.bar(labels, values, color=['#ff4757', '#2ed573'], edgecolor='black',
width=0.6)
plt.title('Performance Comparison: Avg Turnaround Time', fontweight='bold')
plt.ylabel('Average Turnaround Time (Minutes)') plt.grid(axis='y',
linestyle='--', alpha=0.5) for i, v in enumerate(values):
plt.text(i, v + 0.3, f"{v:.2f} mins", ha="center", fontweight="bold",
fontsize=12)
plt.show()
```

**Expected Result**
The AI Smart Scheduler should show a lower Average Turnaround Time than Standard Round Robin — proving that dynamic quantum + I/O priority = better performance.

# 4. sklearn.tree & DecisionTreeClassifier

The Smart CPU Scheduler uses machine learning to classify processes as CPU-bound or I/Obound. The tool used for this is DecisionTreeClassifier from Python's scikit-learn (sklearn) library.

## 3.1 What is scikit-learn (sklearn)?

Scikit-learn is a free, open-source machine learning library for Python. It provides simple and efficient tools for data analysis and modeling. It is built on top of NumPy, SciPy, and matplotlib.

| Feature | Description |
|---|---|
| Library Name | scikit-learn (imported as sklearn) |
| Purpose | Machine learning — classification, regression, clustering |
| Language | Python |
| Install Command | pip install scikit-learn |
| Used In This Project | Classifying processes as CPU-bound or I/O-bound |

## 3.2 What is a Decision Tree?

A Decision Tree is a machine learning model that makes decisions by asking a series of yes/no questions about the input data. It looks like a tree structure where each branch represents a decision and each leaf represents an outcome.

**Simple Analogy**
Think of a Decision Tree like a doctor diagnosing a patient: First it asks 'Does the patient have a fever?' → Yes → 'Do they have a cough?' → Yes → 'Diagnosis: Flu'. The same logic applies to our scheduler: it asks questions about a process to decide if it is CPU-bound or I/O-bound.

### 3.3 How DecisionTreeClassifier Works in Our Project

In the Smart CPU Scheduler, the DecisionTreeClassifier is trained to look at process features and predict whether the process is CPU-bound (needs large quantum) or I/O-bound (needs small quantum).

**Training Data Example**

```
from sklearn.tree import DecisionTreeClassifier


# Features: [burst_time, io_frequency, priority]
X_train = [
    [10, 1, 3],    # CPU-bound: long burst, low I/O
    [2,  8, 1],    # I/O-bound: short burst, high I/O
    [8,  2, 2],    # CPU-bound
    [3,  7, 1],    # I/O-bound
]
# Labels: 0 = CPU-bound, 1 = I/O-bound y_train = [0, 1, 0, 1]
```

```
# Create and train the model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

```
# Predict a new process
new_process = [[9, 1, 3]]
prediction = model.predict(new_process)
# Output: [0] → CPU-bound → assign larger quantum
```

**How the Classifier Connects to Scheduling**

| Process Type | Predicted By | Quantum Assigned | Why? |
|---|---|---|---|
| CPU-bound | DecisionTree → 0 | Large (e.g. 8) | Needs long computation |
| I/O-bound | DecisionTree → 1 | Small (e.g. 2) | Finishes quickly, frees CPU |

**Why Use a Decision Tree Here?**

Decision Trees are simple, explainable, and fast — perfect for a real-time scheduler. Unlike complex neural networks, a Decision Tree can be explained to a teacher or examiner step by step, making it ideal for an academic project.

## 5. System Design — Diagrams

The following diagrams illustrate the architecture, data flow, and interactions of the Smart CPU Scheduler system.

### 5.1 Context Diagram (Level 0 DFD)

Shows the system boundary: the Operating System sends process details (Process ID, Arrival Time, Instructions) to the AI-Based Smart CPU Scheduler, which outputs Performance Metrics (Avg Waiting Time, Turnaround Time, CPU Utilization) to the Performance Dashboard.

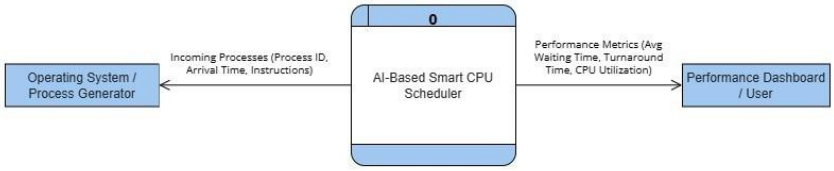


*Figure 1 — Context Diagram: AI-Based Smart CPU Scheduler*

## 5.2 Sequence Diagram

Shows the interaction flow between the User, Scheduler, Process, CPU, and Report. The loop repeats for each process: the scheduler assigns a quantum, the CPU executes the burst, totals are calculated, and a final benchmark report is generated.

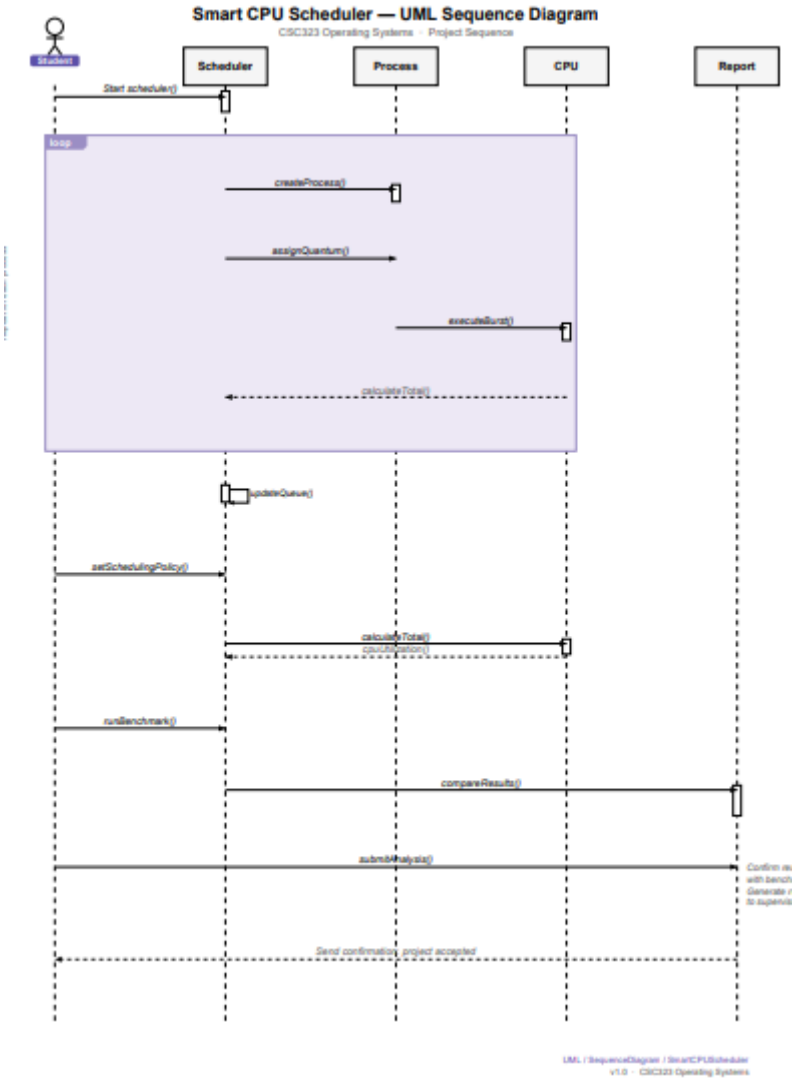


*Figure 2 — UML Sequence Diagram: Scheduler Execution Flow*

## 5.3 Class Diagram

Defines the system classes and their relationships. The abstract Scheduler manages the ReadyQueue and dispatches to the CPU. SmartScheduler extends Scheduler and adds classifyProcess() and assignDynamicQuantum(). BenchmarkReport stores and exports results.

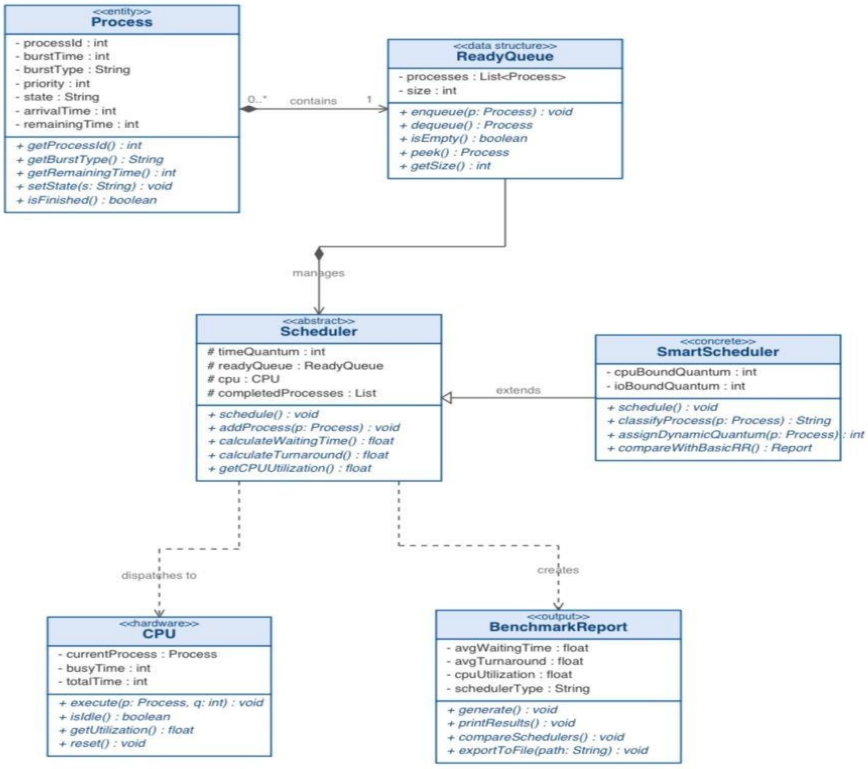


*Figure 3 — UML Class Diagram: System Architecture*

## 5.4 Use Case Diagram

Captures what the User can do with the system: Add Process, View Processes, Delete Process, and Schedule Processes. Scheduling includes two sub-cases: Monitor CPU Usage and Predict Priority using the AI model.

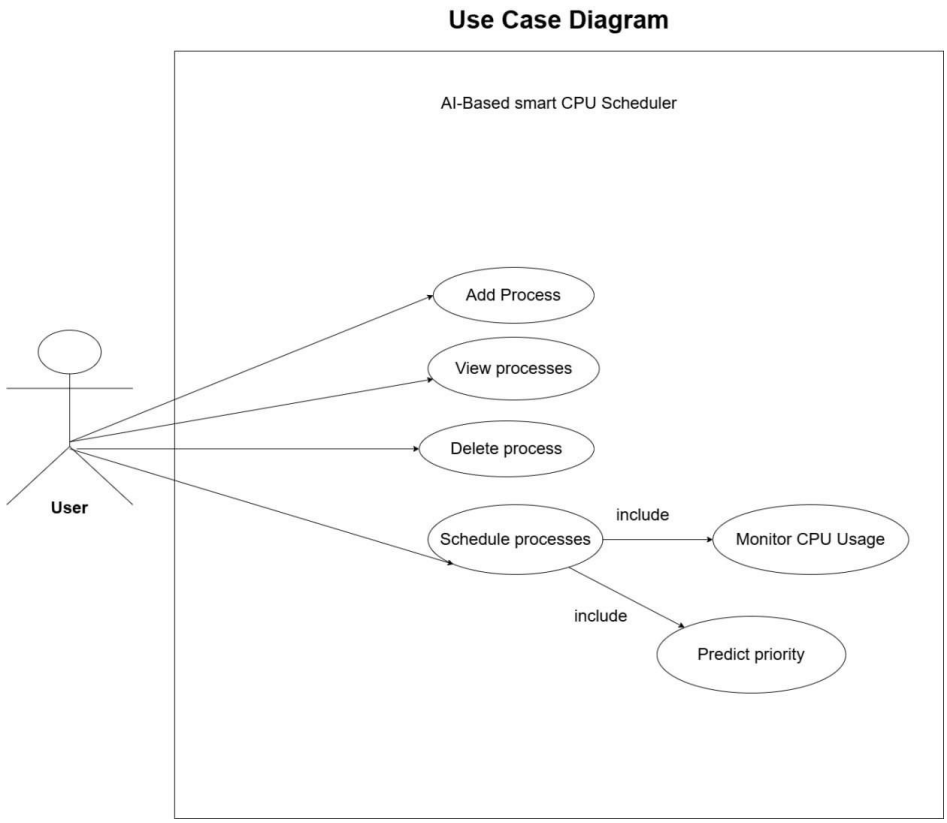


*Figure 4 — Use Case Diagram: AI-Based Smart CPU Scheduler*

## 5.5 Data Flow Diagram (Level 1)

Shows the internal data flow: the Queue Manager feeds the Ready Queue → the AI Prediction Engine classifies processes (CPU/IO Bound) → the Dynamic Quantum Allocator assigns time → the CPU Dispatcher executes → the Metrics Evaluator sends results to the Performance Dashboard.

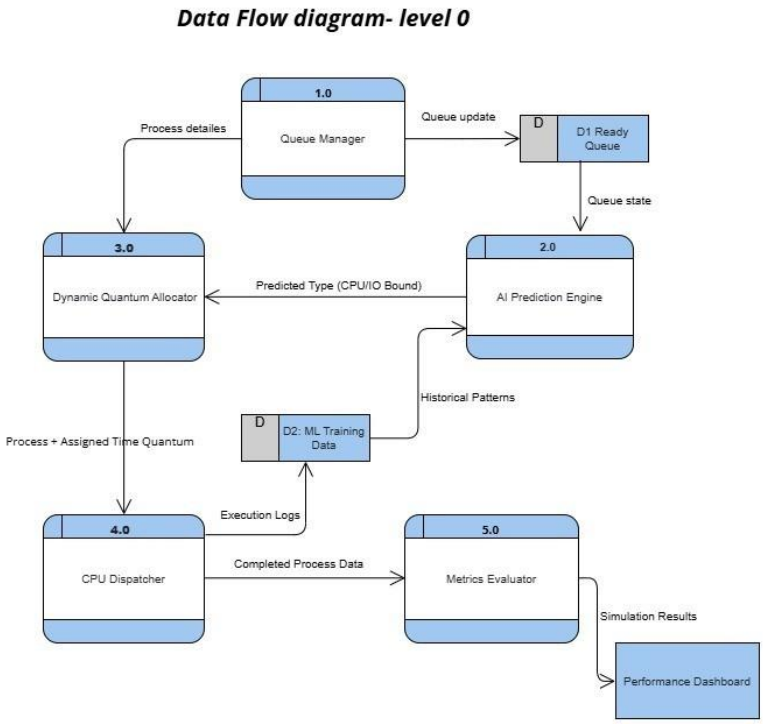


*Figure 5 — Data Flow Diagram Level 1: Internal System Flow*

# 6. Comparative Analysis — Industry Schedulers

To contextualize our Smart CPU Scheduler, we compared it against four major real-world scheduling systems used in industry. This demonstrates the innovation and relevance of our approach.

| System | Scheduling Logic | Key Advantage | Our Project's Value-Add |
|---|---|---|---|
| Apache Spark | DAG / Stage-based | Massively Parallel | Behavioral Prediction (I/O vs CPU) |
| FreeRTOS | Fixed Priority | Strict Deadlines | Dynamic & Adaptive Prioritization |
| Kubernetes | Resource-based | Global Load Balancing | Micro-level Quantum Optimization |
| Linux CFS | Red-Black Tree (Fair) | Balanced Stability | Efficiency over simple fairness |

# 7. Next Steps — Week 8–9

Based on the prototype completed in Weeks 5–7, the following tasks are planned for the Testing & Review phase:

- Implement the Smart Scheduler with dynamic time quantum using DecisionTreeClassifier
- Run side-by-side simulation: Traditional RR vs Smart RR on the same process sets
- Measure and compare: Average Waiting Time, Turnaround Time, CPU Utilization
- Visualize results with charts (bar graphs, Gantt charts)
- Refine and optimize the classifier based on test results

**Week 8–9 Target**
Deliver a working Smart CPU Scheduler that demonstrably reduces average waiting time and improves CPU utilization compared to the Traditional Round Robin prototype built in this phase.

# 8. References


1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
2. GeeksforGeeks. CPU Scheduling in Operating Systems. https://www.geeksforgeeks.org/cpuscheduling-in-operating-systems/
3. Scikit-learn Documentation. sklearn.tree.DecisionTreeClassifier. https://scikit-learn.org/stable/modules/tree.html
4. FreeRTOS.org. (2024). FreeRTOS Scheduling — Real-time kernel fundamentals.
5. Kubernetes.io. (2024). Kubernetes Scheduler Concepts.